You said:

codigo.py
Code

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 7s

ChatGPT said:

Análise de segurança do arquivo `codigo.py`

Veredito: MALICIOSO — alta confiança

Classificação: script de instalação forçada de extensão no Google Chrome, com características de persistência, evasão de mecanismos de integridade e notificação externa.

O código apresenta comportamentos concretos que vão além de uma instalação legítima comum: encerra o Chrome, baixa uma extensão de um endereço externo, modifica diretamente as preferências protegidas do navegador e envia uma notificação a um serviço externo informando a instalação.

Recomendação imediata: não execute o arquivo. Se já foi executado, considere o perfil do Chrome potencialmente comprometido até verificar a extensão instalada e suas permissões.

A análise abaixo é baseada no conteúdo de `codigo.py`, linha por linha por blocos funcionais. Não executei o script. É importante distinguir o que o código comprova daquilo que ainda precisaria ser investigado, como o conteúdo da extensão baixada e a eventual exfiltração de dados.

1. Visão geral do funcionamento

O fluxo principal do programa pode ser representado assim:

1. Encerra o Google Chrome
Força o fechamento dos processos encontrados.

↓

2. Identifica o usuário do Windows
Obtém o nome e o SID da conta.

↓

3. Baixa e extrai uma extensão
Utiliza um servidor externo e um arquivo ZIP.

↓

4. Altera o perfil do Chrome
Insere a extensão em Secure Preferences e recalcula MACs.

↓

5. Envia uma notificação externa
Informa o identificador da extensão e o usuário.

Esse conjunto é especialmente preocupante porque combina instalação não convencional, alteração de configurações protegidas e comunicação externa.

2. Análise passo a passo do código

Etapa 1 — Importação de bibliotecas (linhas 4–15)

O programa importa bibliotecas para criptografia, manipulação de arquivos, acesso ao Windows, requisições HTTP e monitoramento de processos. codigo.py

Biblioteca	Função observada	Avaliação
`hmac`, `hashlib`	Cálculo de códigos de integridade criptográfica	Uso compatível com a alteração das preferências protegidas
`win32security`, `win32api`	Consulta de informações da conta Windows	Permite identificar o usuário e seu SID
`requests`	Comunicação HTTP/HTTPS	Usada para baixar a extensão e enviar notificações
`zipfile`, `tempfile`	Extração de arquivos em diretório temporário	Prepara os arquivos da extensão
`psutil`	Enumeração e encerramento de processos	Usada para fechar o Chrome

Resultado: as bibliotecas não são maliciosas por si mesmas. O risco decorre da maneira como são combinadas.

Etapa 2 — Comunicação com serviço externo (linhas 17–30)

A função `notificar_instalacao()` envia uma mensagem a um tópico do serviço `ntfy.sh`.

A mensagem contém:

- O identificador calculado para a extensão.
- O nome do usuário do Windows.
- Uma indicação explícita de que a extensão foi instalada.

O envio é realizado com `requests.post()`. codigo.py

Por que isso é suspeito?

A notificação cria um canal externo para informar a quem controla ou acompanha aquele tópico que uma instalação foi realizada.

O código não demonstra, por si só, que senhas ou cookies são enviados para esse serviço. O que ele envia explicitamente é o identificador da extensão e o nome de usuário.

Etapa 3 — Encerramento forçado do Chrome (linhas 32–47)

A função `fechar_chrome()` percorre os processos em execução, procura nomes que contenham `chrome` e chama `proc.terminate()`. Depois, aguarda até cinco segundos pelos processos terminarem. codigo.py

Isso pode ser usado para liberar arquivos antes de modificar configurações do navegador.

O comportamento é relevante porque o programa não se limita a preparar uma extensão para instalação manual: ele encerra o navegador antes de modificar seu perfil diretamente.

Classificação: comportamento suspeito, especialmente quando combinado com as etapas seguintes.

Etapa 4 — Identificação do usuário e do SID (linhas 117–128)

O código executa:

- `win32api.GetUserName()`
- `win32security.LookupAccountName()`

- `win32security.ConvertSidToStringSid()`
- `os.getlogin()`

Em seguida, manipula o SID obtido. codigo.py

O SID é um identificador de segurança do Windows. Nesse script, essas informações são usadas durante a manipulação das preferências protegidas do Chrome.

A obtenção dessas informações pode ter usos legítimos. Entretanto, neste contexto, ela ajuda o programa a modificar os dados do perfil de um usuário específico.

Etapa 5 — Download e extração da extensão (linhas 102–115 e 130)

Este é um dos pontos críticos.

O programa baixa um arquivo ZIP de:

`https://antiddos-protection.net/_____________/chrome_extension.zip`

Em seguida, salva o conteúdo em um diretório temporário e extrai o arquivo sem uma verificação explícita de assinatura, hash confiável ou procedência. codigo.py

A URL é usada diretamente pela função principal. codigo.py

Os riscos são:

- O conteúdo executável da extensão não está incluído no arquivo Python analisado.
- O código não verifica se o ZIP corresponde a uma versão aprovada.
- O conteúdo remoto pode mudar sem que o script seja alterado.
- A extensão pode solicitar e utilizar as permissões concedidas no perfil do navegador.

Não é possível confirmar a finalidade real da extensão sem analisar os arquivos extraídos, especialmente `manifest.json` e os scripts JavaScript.

Etapa 6 — Construção do identificador da extensão (linhas 84–89)

A função `get_extension_id()` calcula o SHA-256 do caminho local da extensão, codificado em UTF-16 little-endian, e transforma parte do resultado em caracteres. codigo.py

Esse comportamento é incomum em uma instalação convencional: o identificador de uma extensão do Chrome normalmente deriva da chave pública da extensão, não simplesmente do caminho do diretório.

Isso levanta uma questão técnica importante: o identificador calculado pode não corresponder ao identificador esperado pelo Chrome para aquele pacote. Portanto, **o sucesso da instalação não está garantido**.

Esse detalhe não torna o script benigno; mostra que ele tenta realizar uma instalação por um mecanismo não convencional.

Etapa 7 — Definição de permissões elevadas para a extensão (linhas 136–159)

O script constrói manualmente um objeto JSON para registrar a extensão. Entre as permissões declaradas estão:

`cookies`

Permissão relacionada ao acesso a cookies, que podem conter sessões autenticadas ou outros dados sensíveis.

`tabs`

Permite acesso a informações relacionadas às abas, incluindo URLs conforme o contexto e as permissões aplicáveis.

`scripting`

Permite executar ou injetar scripts em páginas nos contextos autorizados.

`<all_urls>`

Solicita um escopo amplo de acesso a sites. A capacidade efetiva depende das permissões e restrições do Chrome.

As permissões e os campos de configuração são definidos no JSON criado pelo próprio programa. codigo.py

Há ainda `incognito: true`, que indica uma configuração relacionada ao uso em modo anônimo, e `withholding_permissions: false`, que indica que as permissões não estão sendo marcadas como retidas nesse registro.

Avaliação: o conjunto de permissões cria potencial para acesso a informações sensíveis de navegação. Contudo, a declaração das permissões não prova que a extensão efetivamente rouba cookies; isso depende do comportamento do código da extensão baixada.

Etapa 8 — Alteração direta de `Secure Preferences` (linhas 161–174)

Esta é uma das evidências mais fortes de comportamento malicioso.

O script abre o arquivo:

```
C:\Users\<usuário>\AppData\Local\Google\Chrome\User Data\Default\Secure Preferences
```

Depois, insere a extensão na estrutura `extensions.settings` e calcula um HMAC para registrar o valor em `protection.macs`. codigo.py

O arquivo `Secure Preferences` contém configurações protegidas do Chrome. Em vez de utilizar o fluxo normal de instalação do navegador, o script modifica esse arquivo diretamente.

O uso de HMAC, isoladamente, é legítimo. O que torna esse caso preocupante é o propósito observado: recalcular os valores de integridade para que a alteração manual pareça consistente com os mecanismos de proteção do perfil.

O script também contém um valor de chave (`seed`) embutido no código. Não há, nessa parte, uma verificação de autorização do usuário para instalar a extensão.

Interpretação: o programa tenta contornar o fluxo normal de instalação e os mecanismos de integridade das preferências do navegador. Isso é compatível com uma técnica de instalação forçada ou persistência de extensão.

Há uma ressalva técnica: não executei o código no Chrome para confirmar que os cálculos são aceitos pela versão instalada. A implementação pode falhar, inclusive por diferenças na estrutura das preferências ou na validação de integridade.

Etapa 9 — Ativação do modo de desenvolvedor e recálculo dos MACs (linhas 176–189)

O programa define:

```
data["extensions"]["ui"]["developer_mode"] = True
```

Depois, calcula outro HMAC e atualiza o `super_mac`. codigo.py

A sequência é significativa:

1. Insere uma extensão no registro de configurações.
2. Recalcula o MAC associado à extensão.
3. Ativa o modo de desenvolvedor.

4. Recalcula o MAC associado a essa preferência.
5. Atualiza o MAC global de proteção.

A ativação do modo de desenvolvedor não é maliciosa por si só. Porém, nesse contexto, ela faz parte de um procedimento automatizado de alteração do perfil.

O objetivo aparente é permitir o registro de uma extensão fora do fluxo habitual da Chrome Web Store, mantendo os valores de integridade coerentes com as modificações realizadas.

Etapa 10 — Execução automática da rotina principal (linhas 192–196)

O bloco final chama `add_extension()` quando o arquivo Python é executado diretamente. Ao concluir a tentativa de instalação, o programa envia a notificação externa. codigo.py

Não há um mecanismo de confirmação explícita do usuário antes das ações principais. O script foi estruturado para realizar o processo de forma automatizada.

3. Indicadores de comprometimento

Indicador	Evidência	Gravidade
Download de conteúdo externo	Arquivo ZIP obtido de um domínio externo	Alta
Instalação não convencional	Modificação direta de `Secure Preferences`	Alta
Permissões sensíveis	Cookies, abas, scripting e acesso amplo a sites	Alta
Alteração de mecanismos de integridade	Recálculo dos MACs e do `super_mac`	Alta
Comunicação externa	Notificação com identificador e nome do usuário	Média/alta
Encerramento do navegador	Finalização de processos do Chrome antes da alteração	Média/alta

Esses indicadores, considerados em conjunto, sustentam a classificação de alto risco. Nenhum deles, isoladamente, comprova roubo de credenciais, mas a combinação é fortemente suspeita.

4. O que ainda não foi comprovado

Para manter a análise tecnicamente rigorosa, há limitações importantes:

- **Roubo de senhas:** não há código explícito para capturar senhas no arquivo Python analisado.
- **Exfiltração de cookies:** o script registra permissões relacionadas a cookies, mas não contém uma rotina explícita de envio desses cookies.
- **Execução bem-sucedida:** não foi comprovado que a extensão é instalada com sucesso em uma máquina real.
- **Conteúdo da extensão:** o ZIP remoto não foi obtido e analisado nesta avaliação.
- **Reputação do domínio:** não foi realizada uma verificação independente de reputação ou de infraestrutura do endereço.

O ponto decisivo é que o arquivo Python funciona como um instalador ou modificador de perfil para uma extensão cuja finalidade real permanece desconhecida. É possível que a carga remota execute comportamentos mais perigosos do que os visíveis no script.

5. Como agir se o arquivo já foi executado

1. **Interrompa a execução e evite usar o perfil afetado para acessar contas sensíveis.**
2. No Chrome, abra `chrome://extensions` e identifique extensões desconhecidas. Não considere a simples remoção suficiente se houver suspeita de comprometimento persistente.
3. Faça uma verificação completa do sistema com o Microsoft Defender ou outra ferramenta antimalware confiável.
4. Se houver suspeita de acesso a sessões autenticadas, altere as senhas importantes a partir de um dispositivo confiável e encerre as sessões ativas dessas contas.
5. Preserve o arquivo original e, se possível, os registros de segurança. Evite executar novamente o script para testar seu funcionamento.

Conclusão final

Classificação final: MALWARE / INSTALADOR MALICIOSO PROVÁVEL

Grau de confiança: alto quanto ao caráter malicioso ou não autorizado do comportamento; indeterminado quanto à carga final.

A conclusão se baseia principalmente na tentativa de instalar uma extensão remotamente por meio de alterações diretas nas preferências protegidas do Chrome, no uso de permissões sensíveis e na comunicação externa após a instalação.

Não é possível afirmar, com o arquivo isolado, qual família de malware seria responsável nem confirmar que houve roubo de dados. Para isso, seria necessário analisar também o ZIP da extensão e seus scripts.

Minha recomendação é tratar `codigo.py` como malicioso e não permitir sua execução em um computador de uso real.

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

A **categoria principal** desse malware é **Trojan (Remote Payload Installer / Browser Extension Installer)** — um trojan cujo objetivo principal é **instalar furtivamente uma extensão maliciosa no Google Chrome**.

Ele também apresenta características secundárias de **persistência** e **manipulação de configurações do navegador**, mas sua função central é atuar como um **dropper/installer** para uma carga útil hospedada remotamente.

Classificação

Categoria	Aplica?	Evidência no código
Trojan	Sim (principal)	Executa ações ocultas que o usuário normalmente não autorizaria.
Dropper/Installer	Sim	Baixa `chrome_extension.zip` de um servidor externo e o instala.
Browser Hijacker	Parcial	Modifica diretamente as configurações do Chrome (`Secure Preferences`).
Persistence	Sim	Recalcula `MACs` e `super_mac` para manter a alteração persistente.
Spyware	Potencial	Concede permissões como `cookies` e `<all_urls>` , mas o roubo de dados depende da extensão baixada.

Por que é um Trojan?

Ele se enquadra como Trojan porque:

- **Fecha o Chrome** para liberar o perfil do navegador.

- **Baixa uma carga remota** (`chrome_extension.zip`) de um domínio externo.
- **Instala a extensão sem usar o fluxo normal do Chrome**, alterando diretamente `Secure Preferences` .
- **Recalcula os mecanismos de integridade** (`protection.macs e super_mac`) para tentar fazer a alteração parecer legítima.
- **Notifica um servidor externo** (`ntfy.sh`) quando a instalação termina.

Nome mais específico

Se fosse descrito em um relatório de resposta a incidentes, eu o classificaria como:

> **Trojan.Dropper.BrowserExtension (Chrome Secure Preferences Manipulation)**

Essa descrição destaca tanto a função de **dropper** (baixar e instalar a carga útil) quanto a técnica específica usada para modificar as preferências protegidas do Chrome.

Posso também fazer uma ↪ classificação MITRE ATT&CK ou extrair os ↪ IOCs do arquivo.